Contents

Introduction

This document contains commonly used code for competitive programming, specifically in rust and written for the 2026 finale in the norwegian olympiad of informatics.

Basics

Rust boilerplate

```rust
use std::convert::TryInto;
use std::io;

fn main() {
  let mut lines = io::stdin().lines();
}
```

Capturing input

We will use the stdin lines iterator defined above to read the input:

```rust
let n: usize = lines.next().unwrap().unwrap().parse();
```

For reading multiple variables in a single line, one can do the following:

```rust
let [n, q]: [usize; 2] = lines
    .next()
    .unwrap()
```

```
    .unwrap()
    .trim()
    .split_whitespace()
    .map(|x| x.parse().unwrap())
    .collect::<Vec<_>>()
    .try_into()
    .unwrap();
```

, which is expandable to any number of values in the same line.

Enums
Can be defined with

```
#[derive(Debug, Clone, Copy, PartialEq, Eq)]
enum Instruction {
    Build,
    Query,
}
```

Sets
Note that most set operations use borrows, with the exception of `insert`.

```
use std::collections::HashSet;

let mut set: HashSet<usize> = HashSet::new();

set.insert(5);
set.insert(6);

assert!(set.contains(&5));
assert!(!set.contains(&7));

set.remove(&6);
```

Hash Maps (TODO)

Time complexity
Depending on the constraints in the problem, different algorithms should be chosen, approximately based on this:

n	Time complexity
1 000 000	$O(n)$
400 000	$O(n \log n)$
1 000	$O(n^2)$
200	$O(n^3)$

20	$O(2^n)$
10	$O(n!)$

Data structures

Graphs

I like to define graphs as follows:

```
type Graph = Vec<Vec<(usize, usize)>>;
```

Which will be indexed like this:

```
graph[from_node] = [(neighbor1, cost), (neigbor2, cost), (neighbor3, cost)]
```

Here is an example of how a graph can be constructed in rust (taken from my solution to `nio21-finale-togtur`). In this case I am creating an undirected graph. For a directed graph, one would remove the line with `graph[b].push((a, k))`.

```
let mut graph: Vec<Vec<(usize, usize)>> = vec![Vec::new(); n];
for _ in 0..m {
    let [a, b, k]: [usize; 3] = lines
        .next()
        .unwrap()
        .unwrap()
        .trim()
        .split_whitespace()
        .map(|x| x.parse().unwrap())
        .collect::<Vec<_>>()
        .try_into()
        .unwrap();
    graph[a].push((b, k));
    graph[b].push((a, k));
}
```

Dijkstra

Dijkstra's algorithm is an algorithm for finding the lowest total weight path between two nodes in a graph. It has a time complexity of $O((V+E)\log V)$. It requires the following imports:

```
use std::cmp::Reverse;
use std::collections::BinaryHeap;
```

Note

We use `Reverse` here because rust's `BinaryHeap` sorts the values descending, but for Dijkstra we want it to be ascending. `Reverse` is used to reverse the natural ordering of values.

And can be implemented as such:

```
let n: usize; // Number of nodes in the graph
let graph: Graph;
let start: usize;
let target: usize;

let mut heap = BinaryHeap::new();
heap.push(Reverse((0, start)));
let mut dist = vec![usize::MAX; n];
dist[start] = 0;

let mut result = None;

while let Some(Reverse((cost, node))) = heap.pop() {
    if cost > dist[node] {
        continue;
    }

    if node == target {
        result = Some(cost);
        break;
    }

    for &(neighbor, c) in &graph[node] {
        let new_cost = cost + c;
        if new_cost < dist[neighbor] {
            dist[neighbor] = new_cost;
            heap.push(Reverse((new_cost, neighbor)));
        }
    }
}

println!("{}", result.unwrap());
```

There are often problems which require a variation of Dijkstra, for example the aforementioned `nio21-finale-togtur` which adds another layer for each “free edge”. This can easily be implemented by extending the dist DP table and adding another item to the tuple in the heap:

```
heap.push(Reverse((0, v, start))); // (cost, tickets, node)
let mut dist = vec![vec![usize::MAX; v + 1]; n];
dist[start][v] = 0;

// ---

for &(n, c) in &graph[node] {
    let new_cost = cost + c;
    if new_cost < dist[n][tickets] {
        dist[n][tickets] = new_cost;
        heap.push(Reverse((new_cost, tickets, n)));
    }
```

```
    if tickets > 0 && cost < dist[n][tickets - 1] {
        dist[n][tickets - 1] = cost;
        heap.push(Reverse((cost, tickets - 1, n)));
    }
}
```

BFS (TODO)

DFS (TODO)

DP (TODO)

Trees (TODO)

Difference arrays

Difference arrays provide a more efficient way to update all values within an interval. Instead of looping over all items in the interval $[a, b]$, one instead just sets the values at a and b, then passes over the array afterwards. A boolean implementation could look like this (taken from my solution to `nio23-runde2-lynnedslag`):

```
let mut houses: Vec<bool> = vec![false; n];

for _ in k {
  let [a, b]: [usize; 2];
  houses[a] = !houses[a];
  houses[b+1] = !houses[b+1];
}

let mut flip = false;
let mut result = 0;

for h in houses {
  flip ^= h;

  if !flip {
    result += 1;
  }
}
```

Note

We use $b + 1$ instead of b because the interval is inclusive. The difference array marks where the effect starts and stops, and the prefix accumulation applies the effect from a through b.

A more general implementation could look like this:

```
let mut xs: Vec<isize> = vec![0; 10 + 1];
```

```
// Add 3 to the interval [2, 5]
xs[2] += 3;
xs[5+1] -= 3;

// Subtract 9 in the interval [6, 9]
xs[6] -= 9;
xs[9+1] += 9;

let mut d = 0;
let mut result = 0;

for x in xs {
  d += x;
  result += d;
}
```

Note that the previous version is actually a special case of this, in which one would use `+= x % 2` (which I have simplified to using booleans).

Algorithms

Binary search (TODO)